CHAPTER 7

Random-Number Generators

Recommended sections for a first reading: 7.1, 7.2, 7.3.1, 7.3.2, 7.4.1, 7.4.3

7.1 INTRODUCTION

A simulation of any system or process in which there are inherently random components requires a method of generating or obtaining numbers that are *random*, in some sense. For example, the queueing and inventory models of Chaps. 1 and 2 required interarrival times, service times, demand sizes, etc., that were "drawn" from some specified distribution, such as exponential or Erlang. In this and the next chapter, we discuss how random values can be conveniently and efficiently generated from a desired probability distribution for use in executing simulation models. So as to avoid speaking of "generating random variables," which would not be strictly correct since a random variable is defined in mathematical probability theory as a function satisfying certain conditions, we will adopt more precise terminology and speak of "generating random *variates*."

This entire chapter is devoted to methods of generating random variates from the uniform distribution on the interval [0, 1]; this distribution was denoted by U(0, 1) in Chap. 6. Random variates generated from the U(0, 1) distribution will be called *random numbers*. Although this is the simplest continuous distribution of all, it is extremely important that we be able to obtain such independent random numbers. This prominent role of the U(0, 1) distribution stems from the fact that random variates from all other distributions (normal, gamma, binomial, etc.) and realizations of various random processes (e.g., a nonstationary Poisson process) can be obtained by transforming IID random numbers in a way determined by the desired distribution or process. This chapter discusses ways to obtain independent random numbers, and

the following chapter treats methods of transforming them to obtain variates from other distributions, and realizations of various processes.

The methodology of generating random numbers has a long and interesting history; see Hull and Dobell (1962), Morgan (1984, pp. 51–56), and Dudewicz (1975) for entertaining accounts. The earliest methods were essentially carried out by hand, such as casting lots (Matthew 27 : 35), throwing dice, dealing out cards, or drawing numbered balls from a "well-stirred urn." Many lotteries are still operated in this way, as is well known by American males who were of draft age in the late 1960s and early 1970s. In the early twentieth century, statisticians joined gamblers in their interest in random numbers, and mechanized devices were built to generate random numbers more quickly; in the late 1930s, Kendall and Babington-Smith (1938) used a rapidly spinning disk to prepare a table of 100,000 random digits. Some time later, electric circuits based on randomly pulsating vacuum tubes were developed that delivered random digits at rates of up to 50 per second. One such random-number machine, the Electronic Random Number Indicator Equipment (ERNIE), was used by the British General Post Office to pick the winners in the Premium Savings Bond lottery [see Thomson (1959)]. Another electronic device was used by the Rand Corporation (1955) to generate a table of a million random digits. Many other schemes have been contrived, such as picking numbers "randomly" out of phone books or census reports, or using digits in an expansion of π to 100,000 decimal places. There has been more recent interest in building and testing physical random-number "machines"; for example, Miyatake et al. (1983) describe a device based on counting gamma rays.

As computers (and simulation) became more widely used, increasing attention was paid to methods of random-number generation compatible with the way computers work. One possibility would be to hook up an electronic random-number machine, such as ERNIE, directly to the computer. This has several disadvantages, chiefly that we could not reproduce a previously generated random-number stream exactly. (The desirability of being able to do this is discussed later in this section.) Another alternative would be to read in a table, such as the Rand Corporation table, but this would entail either large memory requirements or a lot of time for relatively slow input operations. (Also, it is not at all uncommon for a modern large-scale simulation to use far more than a million random *numbers*, each of which would require several individual random *digits*.) Therefore, research in the 1940s and 1950s turned to *numerical* or *arithmetic* ways to generate "random" numbers. These methods are sequential, with each new number being determined by one or several of its predecessors according to a fixed mathematical formula. The first such arithmetic generator, proposed by von Neumann and Metropolis in the 1940s, is the famous *midsquare method*, an example of which follows.

EXAMPLE 7.1. Start with a four-digit positive integer Z_0 and square it to obtain an integer with up to eight digits; if necessary, append zeros to the left to make it exactly eight digits. Take the *middle four* digits of this eight-digit number as the next four-digit number, Z_1. Place a decimal point at the left of Z_1 to obtain the first "U(0, 1) random number," U_1. Then let Z_2 be the middle four digits of Z_1^2 and let U_2 be Z_2 with a decimal point at the left, and so on. Table 7.1 lists the first few Z_i's and U_i's for $Z_0 = 7182$ (the first four digits to the right of the decimal point in the number e).

TABLE 7.1
The midsquare method

i	Z_i	U_i	Z_i^2
0	7182	—	51,581,124
1	5811	0.5811	33,767,721
2	7677	0.7677	58,936,329
3	9363	0.9363	87,665,769
4	6657	0.6657	44,315,649
5	3156	0.3156	09,960,336
.	.	.	.
.	.	.	.
.	.	.	.

Intuitively the midsquare method seems to provide a good scrambling of one number to obtain the next, and so we might think that such a haphazard rule would provide a fairly good way of generating random numbers. In fact, it does not work very well at all. One serious problem (among others) is that it has a strong tendency to degenerate fairly rapidly to zero, where it will stay forever. (Continue Table 7.1 for just a few more steps, or try $Z_0 = 1009$, the first four digits from the Rand Corporation tables.) This illustrates the danger in assuming that a good random-number generator will always be obtained by doing something strange and nefarious to one number to obtain the next.

A more fundamental objection to the midsquare method is that it is not "random" at all, in the sense of being unpredictable. Indeed, if we know one number, the next is completely determined since the rule to obtain it is fixed; actually, when Z_0 is specified, the *whole sequence* of Z_i's and U_i's is determined. This objection applies to all arithmetic generators (the only kind we consider in the rest of this chapter), and arguing about it usually leads one quickly into mystical discussions about the true nature of truly random numbers. (Sometimes arithmetic generators are called *pseudorandom*, an awkward term that we avoid, even though it is probably more accurate.) Indeed, in an oft-quoted quip, John von Neumann (1951) declared that:

> Any one who considers arithmetical methods of producing random digits is, of course, in a state of sin. For, as has been pointed out several times, there is no such thing as a random number—there are only methods to produce random numbers, and a strict arithmetic procedure of course is not such a method. . . . We are here dealing with mere "cooking recipes" for making digits. . . .

It is seldom stated, however, that von Neumann goes on in the same paragraph to say, less gloomily, that these "recipes"

> . . . probably . . . can not be justified, but should merely be judged by their results. Some statistical study of the digits generated by a given recipe should be made, but exhaustive tests are impractical. If the digits work well on one problem, they seem usually to be successful with others of the same type.

This more practical attitude was shared by Lehmer (1951), who developed what is probably still the most widely used class of techniques for random-number

generation (discussed in Sec. 7.2); he viewed the idea of an arithmetic random-number generator as

> . . . a vague notion embodying the idea of a sequence in which each term is unpredictable to the uninitiated and whose digits pass a certain number of tests traditional with statisticians and depending somewhat on the use to which the sequence is to be put.

More formal definitions of "randomness" in an axiomatic sense are cited by Ripley (1987, p. 19); Niederreiter (1978) argues that statistical randomness may not even be desirable, and that other properties of the generated numbers, such as "evenness" of the distribution of points, are more important in some applications, such as Monte Carlo integration. We agree with most writers that arithmetic generators, if designed carefully, can produce numbers that *appear* to be independent draws from the U(0, 1) distribution, in that they pass a series of statistical tests (see Sec. 7.4). This is a useful definition of "random numbers," to which we subscribe.

A "good" arithmetic random-number generator should possess several properties:

1. Above all, the numbers produced should appear to be distributed uniformly on [0, 1] and should not exhibit any correlation with each other; otherwise, the simulation's results may be completely invalid.
2. From a practical standpoint, we would naturally like the generator to be fast and avoid the need for a lot of storage.
3. We would like to be able to reproduce a given stream of random numbers exactly, for at least two reasons. First, this can sometimes make debugging or verification of the computer program easier. More important, we might want to use *identical* random numbers in simulating different systems in order to obtain a more precise comparison; Sec. 11.2 discusses this in detail.
4. There should be provision in the generator for easily producing separate "streams" of random numbers. As we shall see, a stream is simply a subsegment of the numbers produced by the generator, with one stream beginning where the previous stream ends. We can think of the different streams as being separate and independent generators (provided that we do not use up a whole stream, whose length is typically chosen to be a very large number). Thus, the user can "dedicate" a particular stream to a particular source of randomness in the simulation. We did this, for example, in the single-server queueing model of Sec. 2.4, where stream 1 was used for generating interarrival times and stream 2 for generating service times. Using separate streams for separate purposes facilitates reproducibility and comparability of simulation results. While this idea has obvious intuitive appeal, there is probabilistic foundation in support of it as well, as discussed in Sec. 11.2. Further advantages of having streams available are discussed in other parts of Chap. 11. The ability to create separate streams for a generator is facilitated if there is an efficient way to jump from the ith random number to the $(i + k)$th random number for large values of k.
5. We would like the generator to be portable, i.e., to produce the same sequence of random numbers (at least up to machine accuracy) for all standard compilers and computers (see Sec. 7.2.2).

Most of the commonly used generators are quite fast, require very little storage, and can easily reproduce a given sequence of random numbers, so that points 2 and 3 above are almost universally met. Furthermore, most generators now have the facility for multiple streams in some way, especially those generators included in modern simulation packages, satisfying point 4. Unfortunately, there are also some generators that fail to satisfy the uniformity and independence criteria of point 1 above, which are absolutely necessary if one hopes to obtain correct simulation results. For example, L'Ecuyer and Simard (2007) report several instances of published generators' displaying very poor performance.

In Sec. 7.2 we discuss the most common kind of generator, while Sec. 7.3 discusses some alternative methods. Section 7.4 discusses how one can test a given random-number generator for the desired statistical properties. Finally, Apps. 7A and 7B contain portable computer code for two random-number generators in C. The first generator was used for the examples in Chaps. 1 and 2. The second generator is known to have better statistical properties and is recommended for real-world applications.

The subject of random-number generation is a complicated one, involving such disparate disciplines as abstract algebra and number theory, on one hand, and systems programming and computer hardware engineering, on the other. General references on random-number generators are the books by Fishman (1996, 2001, 2006), Gentle (2010), Knuth (1998a), and Tezuka (1995) and, in addition, the book chapters by L'Ecuyer (2006, 2012).

7.2 LINEAR CONGRUENTIAL GENERATORS

Many random-number generators in use today are *linear congruential generators* (LCGs), introduced by Lehmer (1951). A sequence of integers $Z_1, Z_2, \ldots$ is defined by the recursive formula

$$Z_i = (aZ_{i-1} + c)(\text{mod } m) \tag{7.1}$$

where m (the *modulus*), a (the *multiplier*), c (the *increment*), and Z_0 (the *seed*) are nonnegative integers. Thus, Eq. (7.1) says that to obtain Z_i, divide $aZ_{i-1} + c$ by m and let Z_i be the *remainder* of this division. Therefore, $0 \le Z_i \le m - 1$, and to obtain the desired random numbers U_i (for $i = 1, 2, \ldots$) on [0, 1], we let $U_i = Z_i/m$. We shall concentrate our attention for the most part on the Z_i's, although the precise nature of the division of Z_i by m should be paid attention to due to differences in the way various computers and compilers handle floating-point arithmetic. In addition to nonnegativity, the integers m, a, c, and Z_0 should satisfy $0 < m$, $a < m$, $c < m$, and $Z_0 < m$.

Immediately, two objections could be raised against LCGs. The first objection is one common to all (pseudo) random-number generators, namely, that the Z_i's defined by Eq. (7.1) are not really random at all. In fact, one can show by mathematical induction that for $i = 1, 2, \ldots,$

$$Z_i = \left[a^i Z_0 + \frac{c(a^i - 1)}{a - 1} \right] (\text{mod } m)$$

so that *every* Z_i is completely determined by m, a, c, and Z_0. However, by careful choice of these four parameters we try to induce behavior in the Z_i's that makes the corresponding U_i's *appear* to be IID U(0, 1) random variates when subjected to a variety of tests (see Sec. 7.4).

The second objection to LCGs might be that the U_i's can take on only the rational values $0, 1/m, 2/m, \ldots, (m-1)/m$; in fact, the U_i's might actually take on only a fraction of these values, depending on the specification of the constants m, a, c, and Z_0, as well as on the nature of the floating-point division by m. Thus there is no possibility of getting a value of U_i between, say, $0.1/m$ and $0.9/m$, whereas this *should* occur with probability $0.8/m > 0$. As we shall see, the modulus m is usually chosen to be very large, say 10^9 or more, so that the points in [0, 1] where the U_i's can fall are very dense; for $m \geq 10^9$, there are at least a billion possible values.

EXAMPLE 7.2. Consider the LCG defined by $m = 16$, $a = 5$, $c = 3$, and $Z_0 = 7$. Table 7.2 gives Z_i and U_i (to three decimal places) for $i = 1, 2, \ldots, 19$. Note that $Z_{17} = Z_1 = 6$, $Z_{18} = Z_2 = 1$, and so on. That is, from $i = 17$ through 32, we shall obtain *exactly* the same values of Z_i (and hence U_i) that we did from $i = 1$ through 16, and in *exactly* the same order. (We do not seriously suggest that anyone use this generator since m is so small; it only illustrates the arithmetic of LCGs.)

The "looping" behavior of the LCG in Example 7.2 is inevitable. By the definition in Eq. (7.1), whenever Z_i takes on a value it has had previously, exactly the same sequence of values is generated, and this cycle repeats itself endlessly. The length of a cycle is called the *period* of a generator. For LCGs, Z_i depends *only* on the previous integer Z_{i-1}, and since $0 \leq Z_i \leq m-1$, it is clear that the period is at most m; if it is in fact m, the LCG is said to have *full period.* (The LCG in Example 7.2 has full period.) Clearly, if a generator is full-period, any choice of the initial seed Z_0 from $\{0, 1, \ldots, m-1\}$ will produce the entire cycle in some order. If, however, a generator has less than full period, the cycle length could in fact depend on the particular value of Z_0 chosen, in which case we should really refer to the period of the *seed* for this generator.

Since large-scale simulation projects can use millions of random numbers, it is manifestly desirable to have LCGs with long periods. Furthermore, it is comforting to have full-period LCGs, since we are assured that every integer between 0 and $m-1$ will occur exactly once in each cycle, which should contribute to the uniformity of the U_i's. (Even full-period LCGs, however, can exhibit nonuniform behavior in segments within a cycle. For example, if we generate only $m/2$ consecutive Z_i's,

TABLE 7.2
The LCG $Z_i = (5Z_{i-1} + 3) \pmod{16}$ with $Z_0 = 7$

i	Z_i	U_i	i	Z_i	U_i	i	Z_i	U_i	i	Z_i	U_i
0	7	—	5	10	0.625	10	9	0.563	15	4	0.250
1	6	0.375	6	5	0.313	11	0	0.000	16	7	0.438
2	1	0.063	7	12	0.750	12	3	0.188	17	6	0.375
3	8	0.500	8	15	0.938	13	2	0.125	18	1	0.063
4	11	0.688	9	14	0.875	14	13	0.813	19	8	0.500

they may leave large gaps in the sequence $0, 1, \ldots, m-1$ of possible values.) Thus, it is useful to know how to choose m, a, and c so that the corresponding LCG will have full period. The following theorem, proved by Hull and Dobell (1962), gives such a characterization.

THEOREM 7.1. The LCG defined in Eq. (7.1) has full period if and only if the following three conditions hold:

(a) The only positive integer that (exactly) divides both m and c is 1.

(b) If q is a prime number (divisible by only itself and 1) that divides m, then q divides $a-1$.

(c) If 4 divides m, then 4 divides $a-1$.

[Condition (a) in Theorem 7.1 is often stated as "c is relatively prime to m."]

Obtaining a full (or at least a long) period is just one desirable property for a good LCG; as indicated in Sec. 7.1, we also want good statistical properties (such as apparent independence), computational and storage efficiency, reproducibility, facilities for separate streams, and portability (see Sec. 7.2.2). Reproducibility is simple, for we must only remember the initial seed used, Z_0, and initiate the generator with this value again to obtain the same sequence of U_i's exactly. Also, we can easily resume generating the Z_i's at any point in the sequence by saving the final Z_i obtained previously and using it as the new seed; this is a common way to obtain nonoverlapping, "independent" sequences of random numbers.

Streams are typically set up in a LCG by simply specifying the initial seed for each stream. For example, if we want streams of length 1,000,000 each, we set Z_0 for the first stream to some value, then use $Z_{1,000,000}$ as the seed for the second stream, $Z_{2,000,000}$ as the seed for the third stream, and so on. Thus, we see that streams are actually nonoverlapping adjacent subsequences of *the single* sequence of random numbers being generated; if we were to use more than 1,000,000 random numbers from one stream in the above example, we would be encroaching on the beginning of the next stream, which might already have been used for something else, resulting in unwanted correlation.

In the remainder of this section we consider the choice of parameters for obtaining good LCGs and identify some poor LCGs that are still in use. Because of condition (a) in Theorem 7.1, LCGs tend to behave differently for $c > 0$ (called *mixed* LCGs) than for $c = 0$ (called *multiplicative* LCGs).

7.2.1 Mixed Generators

For $c > 0$, condition (a) in Theorem 7.1 is possible, so we might be able to obtain full period m, as we now discuss. For a large period and high density of the U_i's on [0, 1], we want m to be large. Furthermore, in the early days of computer simulation when computers were relatively slow, dividing by m to obtain the remainder in Eq. (7.1) was a relatively slow arithmetic operation, and it was desirable to avoid having to do this division explicitly. A choice of m that is good in all these respects is $m = 2^b$, where b is the number of bits (*bi*nary digi*ts*) in a word on the computer being used that are available for actual data storage. For example, most computers

and compilers have 32-bit words, the leftmost bit being a sign bit, so $b = 31$ and $m = 2^{31} > 2.1$ billion. Furthermore, choosing $m = 2^b$ does allow us to avoid explicit division by m on most computers by taking advantage of *integer overflow.* The largest integer that can be represented is $2^b - 1$, and any attempt to store a larger integer W (with, say, $h > b$ bits) will result in loss of the left (most significant) $h - b$ bits of this oversized integer. What remains in the retained b bits is precisely $W \pmod{2^b}$.

With the choice of $m = 2^b$, Theorem 7.1 says that we shall obtain a full period if c is odd and $a - 1$ is divisible by 4. Furthermore, Z_0 can be any integer between 0 and $m - 1$ without affecting the period. We will, however, focus on multiplicative LCGs in the remainder of Sec. 7.2, because they are much more widely used.

7.2.2 Multiplicative Generators

Multiplicative LCGs are advantageous in that the addition of c is not needed, but they cannot have full period since condition (*a*) of Theorem 7.1 cannot be satisfied (because, for example, m is positive and divides both m and $c = 0$). As we shall see, however, it is possible to obtain period $m - 1$ if m and a are chosen carefully.

As with mixed generators, it's still computationally efficient to choose $m = 2^b$ and thus avoid explicit division. However, it can be shown [see, for example, Knuth (1998a, p. 20)] that in this case the period is at most 2^{b-2}, that is, only *one-fourth* of the integers 0 through $m - 1$ can be obtained as values for the Z_i's. (In fact, the period is 2^{b-2} if Z_0 is odd and a is of the form $8k + 3$ or $8k + 5$ for some $k = 0, 1, \ldots$.) Furthermore, we generally shall not know *where* these $m/4$ integers will fall; i.e., there might be unacceptably large gaps in the Z_i's obtained. Additionally, if we choose a to be of the form $2^l + j$ (so that the multiplication of Z_{i-1} by a is replaced by a shift and j adds), poor statistical properties can be induced. The generator usually known as RANDU is of this form ($m = 2^{31}$, $a = 2^{16} + 3 = 65{,}539$, $c = 0$) and has been shown to have very undesirable statistical properties (see Sec. 7.4). Even if one does not choose $a = 2^l + j$, using $m = 2^b$ in multiplicative LCGs is probably not a good idea, if only because of the shorter period of $m/4$ and the resulting possibility of gaps.

Because of these difficulties associated with choosing $m = 2^b$ in multiplicative LCGs, attention was paid to finding other ways of specifying m. Such a method, which has proved to be quite successful, was reported by Hutchinson (1966), who attributed the idea to Lehmer. Instead of letting $m = 2^b$, it was proposed that m be the largest prime number that is less than 2^b. For example, in the case of $b = 31$, the largest prime that is less than 2^{31} is, very agreeably, $2^{31} - 1 = 2{,}147{,}483{,}647$. Now for m prime, it can be shown that the period is $m - 1$ if a is a *primitive element modulo m*; that is, the smallest integer l for which $a^l - 1$ is divisible by m is $l = m - 1$; see Knuth (1998a, p. 20). With m and a chosen in this way, we obtain each integer $1, 2, \ldots, m - 1$ exactly once in each cycle, so that Z_0 can be any integer from 1 through $m - 1$ and a period of $m - 1$ will still result. These are called *prime modulus multiplicative LCGs* (PMMLCGs).

Two issues immediately arise concerning PMMLCGs: (1) How does one obtain a primitive element modulo m? Although Knuth (1998a, pp. 20–21) gives some

characterizations, the task is quite complicated from a computational standpoint. We shall, in essence, finesse this point by discussing below two widely used PMMLCGs. (2) Since we are not choosing $m = 2^b$, we can no longer use the integer overflow mechanism directly to effect division modulo m. A technique for avoiding explicit division in this case, which also uses a type of overflow, was given by Payne, Rabung, and Bogyo (1969) and has been called *simulated division*. Marse and Roberts' portable generator, which we discuss below, uses simulated division.

Considerable work has been directed toward identifying good multipliers a for PMMLCGs that are primitive elements modulo $m^* = 2^{31} - 1$, which result in a period of $m^* - 1$. In an important set of papers, Fishman and Moore (1982, 1986) evaluated *all* multipliers a that are primitive elements modulo m^*, numbering some 534 million. They used both empirical and theoretical tests (see Sec. 7.4 below), and they identified several multipliers that perform well according to a number of fairly stringent criteria.

Two particular values of a that have been widely used for the modulus m^* are $a_1 = 7^5 = 16{,}807$ and $a_2 = 630{,}360{,}016$, both of which are primitive elements modulo m^*. [However, neither value of a was found by Fishman and Moore to be among the best (see Sec. 7.4.2).] The multiplier a_1 was originally suggested by Lewis, Goodman, and Miller (1969), and it was used by Schrage (1979) in a clever FORTRAN implementation using simulated division. The importance of Schrage's code was that it provided at that time a reasonably good and portable random-number generator.

The multiplier a_2, suggested originally by Payne, Rabung, and Bogyo (1969), was found by Fishman and Moore to yield statistical performance better than does a_1 (see Sec. 7.4.2). Marse and Roberts (1983) provided a highly portable FORTRAN routine for this multiplier, and a C version of this generator is given in App. 7A. This is the generator that we used for all the examples in Chaps. 1 and 2, and it is the one built into the simlib package in Chap. 2.

The PMMLCG with $m = m^* = 2^{31} - 1$ and $a = a_2 = 630{,}360{,}016$ may provide acceptable results for some applications, particularly if the required number of random numbers is not too large. However, many experts [see, e.g., L'Ecuyer, Simard, Chen, and Kelton (2002) and Gentle (2010, p. 21)] recommend that LCGs with a modulus of around 2^{31} should no longer be used as the random-number generator in a general-purpose software package (e.g., for discrete-event simulation). Not only can the period of the generator be exhausted in a few minutes on many computers, but, more importantly, the relatively poor statistical properties of these generators can bias simulation results for sample sizes that are *much smaller* than the period of the generator. For example, L'Ecuyer and Simard (2001) found using the birthday spacings test that the PMMLCGs with modulus m^* and multipliers a_1 or a_2 exhibit a certain departure from what would be expected in a sample of independent observations from the U(0, 1) distribution if the number of observations in the sample is approximately 8 times the *cube root* of the period of the generator. (As the sample size gets larger, the power of the test increases, eventually detecting the fact that the U_i's are not truly uniformly distributed.) Thus, the "safe sample size" of these generators is actually approximately 10,000 [see also L'Ecuyer et al. (2000)].

If a random-number generator with a larger period and better statistical properties is desired, then the combined multiple recursive generator of L'Ecuyer (Sec. 7.3.2), the Mersenne twister (Sec. 7.3.3), or the WELL generator (Sec. 7.3.3) should be considered.

7.3 OTHER KINDS OF GENERATORS

Although LCGs are probably the most widely used and best understood kind of random-number generator, there are many alternative types. (We have already seen one alternative in Sec. 7.1, the midsquare method, which is not recommended.) Most of these other generators have been developed in an attempt to obtain longer periods and better statistical properties. Our treatment in this section is meant not to be an exhaustive compendium of all kinds of generators, but only to indicate some of the main alternatives to LCGs.

7.3.1 More General Congruences

LCGs can be thought of as a special case of generators defined by

$$Z_i = g(Z_{i-1}, Z_{i-2}, \ldots)(\text{mod } m) \tag{7.2}$$

where g is a fixed deterministic function of previous Z_j's. As with LCGs, the Z_i's defined by Eq. (7.2) lie between 0 and $m - 1$, and the U(0, 1) random numbers are given by $U_i = Z_i/m$. [For LCGs, the function g is, of course, $g(Z_{i-1}, Z_{i-2}, \ldots) = aZ_{i-1} + c$.] Here we briefly discuss a few of these kinds of generators and refer the reader to Knuth (1998a, pp. 26–36) or L'Ecuyer (2012) for a more detailed discussion.

One obvious generalization of LCGs would be to let $g(Z_{i-1}, Z_{i-2}, \ldots) = a'Z_{i-1}^2 + aZ_{i-1} + c$, which produces a *quadratic* congruential generator. A special case that has received some attention is when $a' = a = 1$, $c = 0$, and m is a power of 2; although this particular generator turns out to be a close relative of the mid-square method (see Sec. 7.1), it has better statistical properties. Since Z_i still depends only on Z_{i-1} (and not on earlier Z_j's), and since $0 \le Z_i \le m - 1$, the period of quadratic congruential generators is at most m, as for LCGs.

A different choice of the function g is to maintain linearity but to use earlier Z_j's; this gives rise to generators called *multiple recursive generators* (MRGs) and defined by

$$g(Z_{i-1}, Z_{i-2}, \ldots) = a_1Z_{i-1} + a_2Z_{i-2} + \cdots + a_qZ_{i-q} \tag{7.3}$$

where $a_1, a_2, \ldots, a_q$ are constants. A period as large as $m^q - 1$ then becomes possible if the parameters are chosen properly [see Knuth (1998a, pp. 29–30)]. L'Ecuyer, Blouin, and Couture (1993) investigated such generators, and included a generalization of the spectral test (see Sec. 7.4.2) for their evaluation; they also identified several specific generators of this type that perform well, and give portable

implementations. Additional attention to generators with g of the form in Eq. (7.3) used in Eq. (7.2) has focused on g's defined as $Z_{i-1} + Z_{i-q}$, which includes the old Fibonacci generator

$$Z_i = (Z_{i-1} + Z_{i-2})(\text{mod } m)$$

This generator tends to have a period in excess of m but is completely unacceptable from a statistical standpoint; see Prob. 7.12.

7.3.2 Composite Generators

Several researchers have developed methods that take two or more *separate* generators and combine them in some way to generate the final random numbers. It is hoped that this *composite* generator will exhibit a longer period and better statistical behavior than any of the simple generators composing it. The disadvantage in using a composite generator is, of course, that the cost of obtaining each U_i is more than that of using one of the simple generators alone.

Perhaps the earliest kind of composite generators used a second LCG to *shuffle* the output from the first LCG; they were developed by MacLaren and Marsaglia (1965) and extended by Marsaglia and Bray (1968), Grosenbaugh (1969), and Nance and Overstreet (1975). Initially, a vector $\mathbf{V} = (V_1, V_2, \ldots, V_k)$ is filled sequentially with the first k U_i's from the first LCG ($k = 128$ was originally suggested). Then the second LCG is used to generate a random integer I distributed uniformly on the integers $1, 2, \ldots, k$ (see Sec. 8.4.2), and V_I is returned as the first U(0, 1) variate; the first LCG then replaces this Ith location in $\mathbf{V}$ with its next U_i, and the second LCG randomly chooses the next returned random number from this updated $\mathbf{V}$, etc. Shuffling has a natural intuitive appeal, especially since we would expect it to break up any correlation and greatly extend the period. Indeed, MacLaren and Marsaglia obtained a shuffling generator with very good statistical behavior even though the two individual LCGs were quite poor. In a subsequent evaluation of shuffling, Nance and Overstreet (1978) confirm that shuffling one bad LCG by another bad LCG can result in a good composite generator, e.g., by extending the period when used on computers with short word lengths, but that little is accomplished by shuffling a good LCG. In addition, they found that a vector of length $k = 2$ works as well as much larger vectors.

Several variations on this shuffling scheme have been considered; Bays and Durham (1976) and Gebhardt (1967) propose shuffling a generator by itself rather than by another generator. Atkinson (1980) also reported that simply applying a *fixed* permutation (rather than a random shuffling) of the output of LCGs ". . . removes the effect of all but the worst generators, and very appreciably alleviates the damage that even a dreadful generator can cause."

Despite these apparent advantages of rearranging a simple generator's output in some way, there is not as much known about shuffled generators; for example, one cannot jump into an arbitrary point of a shuffled generator's output sequence without generating all the intermediate values, whereas this is possible for LCGs.

Another way to combine two generators is discussed and evaluated by L'Ecuyer (1988) [see also L'Ecuyer and Tezuka (1991)]. In simplified form, the idea is to let $\{Z_{1i}\}$ and $\{Z_{2i}\}$ denote the integer sequences generated by two different LCGs with different moduli, then let $Z_i = (Z_{1i} - Z_{2i})(\text{mod } m)$ for some integer m, and finally set $U_i = Z_i/m$. This idea could clearly be extended to more than two generators, and has several advantages; the period is very long (at least 10^{18} in one example given by L'Ecuyer), the multipliers in each of the component generators can be small (thus promoting portability and usability on virtually any computer), the resulting generator is quite fast, and the statistical properties of these generators also appear to be very good.

One very appealing class of generators is obtained by combining MRGs (discussed in Sec. 7.3.1) in a particular way, and was developed and studied by L'Ecuyer (1996a, 1999a). In general, J different MRGs [using the function g from Eq. (7.3) in the recursion (7.2)] are implemented simultaneously to form the sequences $\{Z_{1,i}\}$, $\{Z_{2,i}\}, \ldots, \{Z_{J,i}\}$. Letting m_1 be the modulus m from Eq. (7.2) used in the first of these MRGs, and letting $\delta_1, \delta_2, \ldots, \delta_J$ be specified constants, we define

$$Y_i = (\delta_1 Z_{1,i} + \delta_2 Z_{2,i} + \cdots + \delta_J Z_{J,i})(\text{mod } m_1)$$

and finally return $U_i = Y_i/m_1$ as the ith random number for use in the simulation. Although the parameters for these kinds of generators must be chosen very carefully, extremely long periods and exceptionally good statistical properties are possible. L'Ecuyer (1999a) carried out an extensive search for good parameter settings and supplied small, portable, and fast C code for several recommended cases; the simplest of these cases, where $J = 2$ MRGs are combined, is defined by

$$\begin{aligned}
Z_{1,i} &= (1{,}403{,}580 Z_{1,i-2} - 810{,}728 Z_{1,i-3})[\text{mod } (2^{32} - 209)] \\
Z_{2,i} &= (527{,}612 Z_{2,i-1} - 1{,}370{,}589 Z_{2,i-3})[\text{mod } (2^{32} - 22{,}853)] \\
Y_i &= (Z_{1,i} - Z_{2,i})[\text{mod } (2^{32} - 209)] \\
U_i &= \frac{Y_i}{2^{32} - 209}
\end{aligned}$$

and has a period of approximately 2^{191} (which is about 3.1×10^{57}) as well as excellent statistical properties through dimension 45 (see Sec. 7.4.2). (This generator is denoted by MRG32k3a in the literature.) Note that for this generator, the seed is actually a 6-vector $(Z_{1,0}, Z_{1,1}, Z_{1,2}, Z_{2,0}, Z_{2,1}, Z_{2,2})$ and the first returned random number would be indexed as U_3. In App. 7B we give a portable C-code implementation for this particular generator, which supports a large number of streams spaced very far apart.

This generator with many large streams and substreams is implemented in the Arena [Kelton et al. (2010, p. 518)], AutoMod [Banks (2004, p. 367)], and WITNESS [Lanner (2013)] simulation packages. (Substreams are nonoverlapping adjacent subsequences of the random numbers in a stream.) If one is making multiple replications of their simulation model, these packages will automatically move to the beginning of the next substream for each stream at the beginning of replications 2, 3, ... This can help synchronize the random numbers across the different system

configurations of interest when using the variance-reduction technique called common random numbers (see Sec. 11.2.3).

Wichmann and Hill (1982) proposed the following idea for combining three generators, again striving for long period, portability, speed, and usability on small computers (as well as statistical adequacy). If U_{1i}, U_{2i}, and U_{3i} are the ith random numbers produced by three separate generators, then let U_i be the fractional part (i.e., ignore any digits to the left of the decimal point) of $U_{1i} + U_{2i} + U_{3i}$; see Prob. 7.13 for the underlying motivation. This indeed produces a very long period [although not as long as claimed in the original paper; see Wichmann and Hill (1984)] and is highly portable and efficient. It was later pointed out by McLeod (1985), however, that their code may have numerical difficulties in some computer architectures. Zeisel (1986) subsequently showed that this generator is identical to a multiplicative LCG, but that this equivalent LCG has a very large modulus and multiplier; thus, the Wichmann-Hill generator turned out to be a way to implement a multiplicative LCG with very large parameters on even the smallest computers. The Wichmann-Hill generator is implemented in Microsoft Excel 2010.

There are many conceivable ways to combine individual random-number generators, some of which we have reviewed above. Others are discussed by Collins (1987), Fishman (1996, pp. 634–645), Gentle (2010, pp. 46–51), and L'Ecuyer (1994b).

7.3.3 Feedback Shift Register Generators

Several interesting kinds of generators have been developed on the basis of a paper by Tausworthe (1965). These generators, which are related to cryptographic methods, operate directly on bits to form random numbers.

Define a sequence $b_1, b_2, \ldots$ of binary digits by the recurrence

$$b_i = (c_1 b_{i-1} + c_2 b_{i-2} + \cdots + c_q b_{i-q}) (\text{mod } 2) \tag{7.4}$$

where $c_1, c_2, \ldots, c_{q-1}$ are constants that are equal to 0 or 1 and $c_q = 1$. Note the similarity between the recurrence for b_i and Eq. (7.3). In most applications of *Tausworthe generators*, only two of the c_j coefficients are nonzero for computational simplicity, in which case Eq. (7.4) becomes

$$b_i = (b_{i-r} + b_{i-q}) (\text{mod } 2) \tag{7.5}$$

for integers r and q satisfying $0 < r < q$. Execution of Eq. (7.5) is expedited by noting that addition modulo 2 is equivalent to the *exclusive-or* instruction on bits. That is, Eq. (7.5) can be expressed as

$$b_i = \begin{cases} 0 & \text{if} \quad b_{i-r} = b_{i-q} \\ 1 & \text{if} \quad b_{i-r} \neq b_{i-q} \end{cases}$$

which is denoted by $b_i = b_{i-r} \oplus b_{i-q}$. To initialize the $\{b_i\}$ sequence, $b_1, b_2, \ldots, b_q$ must be specified somehow; this is akin to specifying the seed Z_0 for LCGs.

To form a sequence of binary integers $W_1, W_2, \ldots$, we string together l consecutive b_i's and consider this as a *number in base 2*. That is,

$$W_1 = b_1 b_2 \cdots b_l$$

and

$$W_i = b_{(i-1)l+1} b_{(i-1)l+2} \cdots b_{il} \qquad \text{for } i = 2, 3, \ldots$$

Note that the recurrence for the W_i's is the same as the recurrence for the b_i's given by Eq. (7.5), namely

$$W_i = W_{i-r} \oplus W_{i-q} \tag{7.6}$$

where the exclusive-or operation is performed bitwise. The ith U(0, 1) random number U_i is then defined by

$$U_i = \frac{W_i}{2^l} \qquad \text{for } i = 1, 2, \ldots$$

The maximum period of the $\{b_i\}$ sequence is $2^q - 1$, since $b_{i-1}, b_{i-2}, \ldots, b_{i-q}$ can take on 2^q different possible states and the occurrence of the q-tuple $0, 0, \ldots, 0$ would cause the $\{b_i\}$ sequence to stay in that state forever. Let

$$f(x) = x^q + c_1 x^{q-1} + \cdots + c_{q-1} x + 1$$

be the characteristic polynomial of the recurrence given by Eq. (7.4). Tausworthe (1965) showed that the period of the b_i's is, in fact, $2^q - 1$ if and only if the polynomial $f(x)$ is *primitive* [see Knuth (1998a, pp. 29–30)] over the Galois field $\mathcal{F}_2$, which is the set $\{0, 1\}$ on which the binary operations of addition and multiplication modulo 2 are defined. If l is relatively prime to $2^q - 1$, then the period of the W_i's (and the U_i's) will also be $2^q - 1$. Thus, for a computer with 31 bits for actual data storage, the maximum period is $2^{31} - 1$, which is the same as that for a LCG.

EXAMPLE 7.3. Let $r = 3$ and $q = 5$ in Eq. (7.5), and let $b_1 = b_2 = \cdots = b_5 = 1$. Thus, for $i \geq 6$, b_i is the "exclusive-or" of b_{i-3} with b_{i-5}. In this case, $f(x)$ is the trinomial $x^5 + x^2 + 1$, which is, in fact, primitive over $\mathcal{F}_2$. The first 40 b_i's are then

$$1111100011011101010000100101100111110001$$

Note that the period of the bits is $31 = 2^5 - 1$, since b_{32} through b_{36} are the same as b_1 through b_5. If $l = 4$ (which is relatively prime to 31), then the following sequence of W_i's is obtained:

$$15, 8, 13, 13, 4, 2, 5, 9, 15, 1, \ldots$$

which also has a period of 31 (see Prob. 7.15). The corresponding U_i's are obtained by dividing the W_i's by $16 = 2^4$.

The original motivation for suggesting that the b_i's be used as a source of U(0, 1) random numbers came from the observation that the recurrence given by Eq. (7.4) can be implemented on a binary computer using a switching circuit called a *linear feedback shift register* (LFSR). This is an array of q bits that is shifted, say, to the left one position at a time, with the bit shifted out on the left combined with other bits in the array to form the new rightmost bit. Because of the relationship between the recurrence (7.4) and a feedback shift register, Tausworthe generators are also called *LFSR generators.*

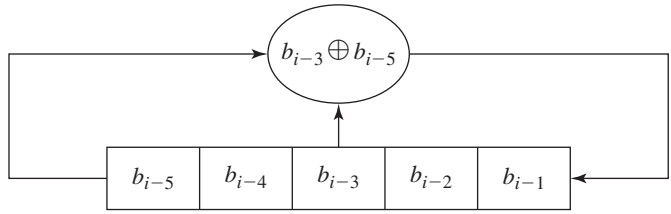


FIGURE 7.1
A LFSR corresponding to the recurrence (7.5) with $r = 3$ and $q = 5$.

EXAMPLE 7.4. The generator discussed in Example 7.3 can be represented by the LFSR shown in Fig. 7.1. The bits b_{i-3} and b_{i-5} are combined using the exclusive-or operation to produce a new bit that goes into the rightmost location (i.e., the one that previously contained bit b_{i-1}) of the array. The bit that was in the leftmost location of the array (i.e., b_{i-5}) is removed from the array. The values of b_{i-5}, b_{i-4}, b_{i-3}, b_{i-2}, and b_{i-1} for $i = 6, 7, \ldots, 15$ are given in Table 7.3.

Unfortunately, LFSR generators are known to have statistical deficiencies, as discussed by Matsumoto and Kurita (1996) and Tezuka (1995). However, L'Ecuyer (1996b, 1999b) considered combined LFSR generators, which have better statistical properties and a larger period.

Lewis and Payne (1973) introduced a modification of the LFSR generator that they called a *generalized feedback shift register* (GFSR) generator. To obtain a sequence of l-bit binary integers $Y_1, Y_2, \ldots$, the sequence of bits $b_1, b_2, \ldots$ produced by Eq. (7.5) is used to fill the first (leftmost) binary position of the integers being formed. Then the same sequence of bits, but with a delay of d, is used to fill the second binary position of the integers. That is, the bits $b_{1+d}, b_{2+d}, \ldots$ are used for the second binary position. Finally, bits $b_{1+(l-1)d}, b_{2+(l-1)d}, \ldots$ are used to fill the lth binary position of the integers. The period of the Y_i's will be $2^q - 1$ if $Y_1, Y_2, \ldots, Y_q$

TABLE 7.3
Successive states of the LFSR for $r = 3$ and $q = 5$

i	$b^{\dagger}_{i-5}$	b_{i-4}	$b^{\dagger}_{i-3}$	b_{i-2}	b_{i-1}
6	1	1	1	1	1
7	1	1	1	1	0
8	1	1	1	0	0
9	1	1	0	0	0
10	1	0	0	0	1
11	0	0	0	1	1
12	0	0	1	1	0
13	0	1	1	0	1
14	1	1	0	1	1
15	1	0	1	0	1

†Source of bits.

TABLE 7.4
GFSR generator with $r = 3, q = 5, l = 4$, and $d = 6$

i	Y_i	i	Y_i	i	Y_i
1	1011 = 11	13	1100 = 12	25	0010 = 2
2	1010 = 10	14	1000 = 8	26	1001 = 9
3	1010 = 10	15	0011 = 3	27	0101 = 5
4	1100 = 12	16	1001 = 9	28	1101 = 13
5	1110 = 14	17	0011 = 3	29	1110 = 14
6	0001 = 1	18	1111 = 15	30	0111 = 7
7	0110 = 6	19	0001 = 1	31	0100 = 4
8	0100 = 4	20	0000 = 0	32	1011 = 11
9	1101 = 13	21	0110 = 6	33	1010 = 10
10	1000 = 8	22	0010 = 2	34	1010 = 10
11	0101 = 5	23	1111 = 15	35	1100 = 12
12	1011 = 11	24	0111 = 7	36	1110 = 14

are linearly independent; that is, $a_1Y_1 + a_2Y_2 + \cdots + a_qY_q = 0$ for $a_j = 0$ or 1 implies that all a_j's are 0. Note also that the Y_i's satisfy the recurrence

$$Y_i = Y_{i-r} \oplus Y_{i-q} \tag{7.7}$$

where the exclusive-or operation is performed bitwise.

The following example illustrates in detail how a GFSR generator works.

EXAMPLE 7.5. For the LFSR generator of Example 7.3, $l = 4$, and a delay of $d = 6$, we give the Y_i's produced by the resulting GFSR generator in Table 7.4. Note that the period of the Y_i's is 31, since Y_{32} through Y_{36} are the same as Y_1 through Y_5. Each of the integers 1 through 15 occurs *twice* and 0 occurs once in the period of length 31.

Because of the parallel nature of the binary positions in the Y_i's, a GFSR generator can have l equal to the word size of the computer, regardless of the relationship between l and q. If $l < q$, then there will be many repeated Y_i's but the period will still be $2^q - 1$. Thus, a very long period can be obtained on an l-bit computer by taking q very large. For example, Fushimi (1990) considered a variant of a GFSR generator that has a period of $2^{521} - 1$.

Matsumoto and Kurita (1992, 1994) introduced the idea of a *twisted GFSR* (TGFSR) *generator*, where the recurrence (7.7) is replaced by

$$\mathbf{Y}_i = \mathbf{Y}_{i-r} \oplus A\mathbf{Y}_{i-q} \tag{7.8}$$

where the $\mathbf{Y}_i$'s are now considered as $l \times 1$ vectors and A is an $l \times l$ matrix, both of which consist of 0s and 1s. [If A is the identity matrix, then recurrence (7.8) is the same as recurrence (7.7).] With suitable choices for r, q, and A, a TGFSR generator can have a maximum period of $2^{ql} - 1$ as compared with $2^q - 1$ for a GFSR generator (both require ql bits to store the state of the generator). Matsumoto and Kurita (1994) discuss how to choose A so that a TGFSR generator has good statistical properties. The *Mersenne twister* [see Matsumoto and Nishimura (1998)], which is denoted by MT19937 in the literature, is a variant of the TGFSR generator with an astonishing period of $2^{19,937} - 1$ as well as excellent statistical properties

through dimension 623 (see Sec. 7.4.2). It has become quite popular in general, and it has also been implemented in the Simio [Simio (2013)] and SIMUL8 [SIMUL8 (2013)] simulation packages. It is called the Mersenne twister because $2^{19,937} - 1$ is a *Mersenne prime*, i.e., a prime number of the form $2^p - 1$. (Note that $m^* = 2^{31} - 1$ is also a prime number.)

Panneton et al. (2006) showed that *if* the Mersenne twister happened to start in an initial state with only a few bits set to 1, then the average value of the U_i's produced over the next few thousand steps is likely to be *much less than 0.5*. They went on to develop a class of similar generators, called *well-equidistributed long-period linear* (WELL), that have a large period (e.g., $2^{19,937}$), excellent statistical properties, and less dependence on a bad initial state. See the survey paper by L'Ecuyer and Panneton (2009).

7.4 TESTING RANDOM-NUMBER GENERATORS

As we have seen in Secs. 7.1 through 7.3, all random-number generators currently used in computer simulation are actually completely deterministic. Thus, we can only hope that the U_i's generated *appear* as if they *were* IID U(0, 1) random variates. In this section we discuss several tests to which a random-number generator can be subjected to ascertain how well the generated U_i's do (or can) resemble values of true IID U(0, 1) random variates.

Most computers have a "canned" random-number generator as part of the available software. Before such a generator is actually used in a simulation, we strongly recommend that one identify exactly what kind of generator it is and what its numerical parameters are. Unless a generator is one of the "good" ones identified (and tested) somewhere in the literature (or is one of the specific generators recommended above), the responsible analyst should subject it (at least) to the empirical tests discussed below.

There are two quite different kinds of tests, which we discuss separately in Secs. 7.4.1 and 7.4.2. *Empirical* tests are the usual kinds of statistical tests and are based on the actual U_i's produced by a generator. *Theoretical* tests are not tests in the statistical sense, but use the numerical parameters of a generator to assess it globally without actually generating any U_i's at all.

7.4.1 Empirical Tests

Perhaps the most direct way to test a generator is to *use* it to generate some U_i's, which are then examined statistically to see how closely they resemble IID U(0, 1) random variates. We discuss four such empirical tests; several others are treated in Knuth (1998a, pp. 41–75), L'Ecuyer et al. (2000), and L'Ecuyer and Simard (2001, 2007).

The first test is designed to check whether the U_i's appear to be uniformly distributed between 0 and 1, and it is a special case of a test we have seen before (in Sec. 6.6.2), the chi-square test with all parameters known. We divide [0, 1]

into k subintervals of equal length and generate $U_1, U_2, \ldots, U_n$. (As a general rule, k should be at least 100 here.) For $j = 1, 2, \ldots, k$, let f_j be the number of the U_i's that are in the jth subinterval, and let

$$\chi^2 = \frac{k}{n} \sum_{j=1}^{k} \left(f_j - \frac{n}{k} \right)^2$$

Then for large n, χ^2 will have an approximate chi-square distribution with $k - 1$ df under the null hypothesis that the U_i's are IID U(0, 1) random variables. Thus, we reject this hypothesis at level α if $\chi^2 > \chi^2_{k-1,1-\alpha}$, where $\chi^2_{k-1,1-\alpha}$ is the upper $1 - \alpha$ critical point of the chi-square distribution with $k - 1$ df. [For the large values of k likely to be encountered here, we can use the approximation

$$\chi^2_{k-1,1-\alpha} \approx (k - 1) \left[1 - \frac{2}{9(k-1)} + z_{1-\alpha} \sqrt{\frac{2}{9(k-1)}} \right]^3$$

where $z_{1-\alpha}$ is the upper $1 - \alpha$ critical point of the N(0, 1) distribution.]

EXAMPLE 7.6. We applied the chi-square test of uniformity to the PMMLCG $Z_i = 630{,}360{,}016 Z_{i-1} (\mathrm{mod}\, 2^{31} - 1)$, as implemented in App. 7A, using stream 1 with the default seed. We took $k = 2^{12} = 4096$ (so that the most significant 12 bits of the U_i's are being examined for uniformity) and let $n = 2^{15} = 32{,}768$. We obtained $\chi^2 = 4141.0$; using the above approximation for the critical point, $\chi^2_{4095,0.95} \approx 4245.0$, so the null hypothesis of uniformity is not rejected at level $\alpha = 0.05$. Therefore, *these particular* 32,768 U_i's produced by this generator do not behave in a way that is significantly different from what would be expected from truly IID U(0, 1) random variables, so far as this chi-square test can ascertain.

Our second empirical test, the *serial test*, is really just a generalization of the chi-square test to higher dimensions. If the U_i's were really IID U(0, 1) random variates, the nonoverlapping d-tuples

$$\mathbf{U}_1 = (U_1, U_2, \ldots, U_d), \qquad \mathbf{U}_2 = (U_{d+1}, U_{d+2}, \ldots, U_{2d}), \qquad \ldots$$

should be IID random *vectors* distributed uniformly on the d-dimensional unit hypercube, $[0, 1]^d$. Divide [0, 1] into k subintervals of equal size and generate $\mathbf{U}_1$, $\mathbf{U}_2, \ldots, \mathbf{U}_n$ (requiring nd U_i's). Let $f_{j_1 j_2 \ldots j_d}$ be the number of $\mathbf{U}_i$'s having first component in subinterval j_1, second component in subinterval j_2, etc. (It is easier to tally the $f_{j_1 j_2 \ldots j_d}$'s than might be expected; see Prob. 7.7.) If we let

$$\chi^2(d) = \frac{k^d}{n} \sum_{j_1=1}^{k} \sum_{j_2=1}^{k} \cdots \sum_{j_d=1}^{k} \left(f_{j_1 j_2 \ldots j_d} - \frac{n}{k^d} \right)^2$$

then $\chi^2(d)$ will have an approximate chi-square distribution with $k^d - 1$ df. [See L'Ecuyer, Simard, and Weggenkittl (2002) for further discussion of the serial test.] The test for d-dimensional uniformity is carried out exactly as for the one-dimensional chi-square test above.

EXAMPLE 7.7. For $d = 2$, we tested the null hypothesis that the pairs (U_1, U_2), $(U_3, U_4), \ldots, (U_{2n-1}, U_{2n})$ are IID random vectors distributed uniformly over the unit square. We used the generator in App. 7A, but starting with stream 2, and generated

$n = 32{,}768$ pairs of U_i's. We took $k = 64$, so that the degrees of freedom were again $4095 = 64^2 - 1$ and the level $\alpha = 0.05$ critical value was the same, 4245.0. The value of $\chi^2(2)$ was 4016.5, indicating acceptable uniformity in two dimensions for the first two-thirds of stream 2 (recall from Sec. 2.3 that the streams are of length 100,000 U_i's, and we used $2n = 65{,}536$ of them here). For $d = 3$, we used stream 3, took $k = 16$ (keeping the degrees of freedom as $4095 = 16^3 - 1$ and the level $\alpha = 0.05$ critical value at 4245.0), and generated $n = 32{,}768$ nonoverlapping *triples of* U_i's. And $\chi^2(3)$ was 4174.5, again indicating acceptable uniformity in three dimensions.

Why should we care about this kind of uniformity in *higher* dimensions? If the individual U_i's are correlated, the distribution of the d-vectors $\mathbf{U}_i$ will deviate from d-dimensional uniformity; thus, the serial test provides an indirect check on the assumption that the individual U_i's are independent. For example, if adjacent U_i's tend to be positively correlated, the pairs (U_i, U_{i+1}) will tend to cluster around the southwest-northeast diagonal in the unit square, and $\chi^2(2)$ should pick this up. Finally, it should be apparent that the serial test for $d > 3$ could require a lot of memory to tally the k^d values of $f_{j_1 j_2 \ldots j_d}$. (Choosing $k = 16$ in Example 7.7 when $d = 3$ is probably not a sufficiently fine division of [0, 1].)

The third empirical test we consider, the *runs* (or *runs-up*) *test*, is a more direct test of the independence assumption. (In fact, it is a test of independence *only*; i.e., we are not testing for uniformity in particular.) We examine the U_i sequence (or, equivalently, the Z_i sequence) for unbroken subsequences of maximal length within which the U_i's increase monotonically; such a subsequence is called a *run up*. For example, consider the following sequence $U_1, U_2, \ldots, U_{10}$: 0.86, 0.11, 0.23, 0.03, 0.13, 0.06, 0.55, 0.64, 0.87, 0.10. The sequence starts with a run up of length 1 (0.86), followed by a run up of length 2 (0.11, 0.23), then another run up of length 2 (0.03, 0.13), then a run up of length 4 (0.06, 0.55, 0.64, 0.87), and finally another run up of length 1 (0.10). From a sequence of n U_i's, we count the number of runs up of length 1, 2, 3, 4, 5, and ≥ 6, and then define

$$r_i = \begin{cases} \text{number of runs up of length } i & \text{for } i = 1, 2, \ldots, 5 \\ \text{number of runs up of length } \geq 6 & \text{for } i = 6 \end{cases}$$

(See Prob. 7.8 for an algorithm to tally the r_i's. For the 10 U_i's above, $r_1 = 2$, $r_2 = 2$, $r_3 = 0$, $r_4 = 1$, $r_5 = 0$, and $r_6 = 0$.) The test statistic is then

$$R = \frac{1}{n} \sum_{i=1}^{6} \sum_{j=1}^{6} a_{ij}(r_i - nb_i)(r_j - nb_j)$$

where a_{ij} is the (i, j)th element of the matrix

$$\begin{bmatrix} 4{,}529.4 & 9{,}044.9 & 13{,}568 & 18{,}091 & 22{,}615 & 27{,}892 \\ 9{,}044.9 & 18{,}097 & 27{,}139 & 36{,}187 & 45{,}234 & 55{,}789 \\ 13{,}568 & 27{,}139 & 40{,}721 & 54{,}281 & 67{,}852 & 83{,}685 \\ 18{,}091 & 36{,}187 & 54{,}281 & 72{,}414 & 90{,}470 & 111{,}580 \\ 22{,}615 & 45{,}234 & 67{,}852 & 90{,}470 & 113{,}262 & 139{,}476 \\ 27{,}892 & 55{,}789 & 83{,}685 & 111{,}580 & 139{,}476 & 172{,}860 \end{bmatrix}$$

and the b_i's are given by

$$(b_1, b_2, \ldots, b_6) = \left(\tfrac{1}{6}, \tfrac{5}{24}, \tfrac{11}{120}, \tfrac{19}{720}, \tfrac{29}{5040}, \tfrac{1}{840}\right)$$

[See Knuth (1998a, pp. 66–69) for derivation of these constants.* The a_{ij}'s given above are accurate to five significant digits.] For large n (Knuth recommends $n \geq 4000$), R will have an approximate chi-square distribution with 6 df, under the null hypothesis that the U_i's are IID random variables.

EXAMPLE 7.8. We subjected stream 4 of the generator in App. 7A to the runs-up test, using $n = 5000$, and obtained $(r_1, r_2, \ldots, r_6) = (808, 1026, 448, 139, 43, 4)$, leading to a value of $R = 9.3$. Since $\chi^2_{6,0.95} = 12.6$, we do not reject the hypothesis of independence at level $\alpha = 0.05$.

The runs-up test can be reversed in the obvious way to obtain a runs-down test; the a_{ij} and b_i constants are the same. Since runs tests look solely for independence (and not specifically for uniformity), it would probably be a good idea to apply a runs test *before* performing the chi-square or serial tests, since the last two tests implicitly assume independence.

The final type of empirical test we consider is a direct way to assess whether the generated U_i's exhibit discernible correlation: Simply compute an estimate of the correlation at lags $j = 1, 2, \ldots, l$ for some value of l. Recall from Sec. 4.3 that the correlation at lag j in a sequence $X_1, X_2, \ldots$ of random variables is defined as $\rho_j = C_j/C_0$, where

$$C_j = \mathrm{Cov}(X_i, X_{i+j}) = E(X_i X_{i+j}) - E(X_i)E(X_{i+j})$$

is the covariance between entries in the sequence separated by j; note that $C_0 = \mathrm{Var}(X_i)$. (It is assumed here that the process is covariance-stationary; see Sec. 4.3.) In our case, we are interested in $X_i = U_i$, and under the hypothesis that the U_i's are uniformly distributed on [0, 1], we have $E(U_i) = \frac{1}{2}$ and $\mathrm{Var}(U_i) = \frac{1}{12}$, so that $C_j = E(U_i U_{i+j}) - \frac{1}{4}$ and $C_0 = \frac{1}{12}$. Thus, $\rho_j = 12E(U_i U_{i+j}) - 3$ in this case. From a sequence $U_1, U_2, \ldots, U_n$ of generated values, an estimate of ρ_j can thus be obtained by estimating $E(U_i U_{i+j})$ directly from U_1, U_{1+j}, U_{1+2j}, etc., to obtain

$$\hat{\rho}_j = \frac{12}{h+1} \sum_{k=0}^{h} U_{1+kj}\, U_{1+(k+1)j} - 3$$

where $h = \lfloor (n-1)/j \rfloor - 1$. Under the further assumption that the U_i's are independent, it turns out [see, for example, Banks et al. (2010, p. 292)] that

$$\mathrm{Var}(\hat{\rho}_j) = \frac{13h + 7}{(h+1)^2}$$

*Knuth, D. E., *The Art of Computer Programming*, Vol. 2, p. 67, © 1998, 1981 Pearson Education, Inc. Reproduced by permission of Pearson Education, Inc. All rights reserved.

Under the null hypothesis that $\rho_j = 0$ and assuming that n is large, it can be shown that the test statistic

$$A_j = \frac{\hat{\rho}_j}{\sqrt{\text{Var}(\hat{\rho}_j)}}$$

has an approximate standard normal distribution. This provides a test of zero lag j correlation at level α, by rejecting this hypothesis if $|A_j| > z_{1-\alpha/2}$. The test should probably be carried out for several values of j, since it could be, for instance, that there is no appreciable correlation at lags 1 or 2, but there is dependence between the U_i's at lag 3, due to some anomaly of the generator.

EXAMPLE 7.9. We tested streams 5 through 10 of the generator in App. 7A for correlation at lags 1 through 6, respectively, taking $n = 5000$ in each case; i.e., we tested stream 5 for lag 1 correlation, stream 6 for lag 2 correlation, etc. The values of $A_1, A_2, \ldots, A_6$ were 0.90, −1.03, −0.12, −1.32, 0.39, and 0.76, respectively, none of which is significantly different from 0 in comparison with the N(0, 1) distribution, at level $\alpha = 0.05$. Thus, the first 5000 values in these streams do not exhibit observable autocorrelation at these lags.

As mentioned above, these are just four of the many possible empirical tests. For example, the Kolmogorov-Smirnov test discussed in Sec. 6.6.2 (for the case with all parameters known) could be applied instead of the chi-square test for one-dimensional uniformity. Also, the birthday spacings test was used by L'Ecuyer and Simard (2001) in their testing of LCGs with modulus $m = 2^{31} - 1$ and multipliers a_1 and a_2 (see Sec. 7.2.2), and it was found that these generators failed the test for a sample size of approximately $n = 10{,}000$. They also applied the test to the generators MRG32k3a and MT19937, and uniformity was not rejected for sample sizes up to and including $n = 262{,}144$. Several empirical tests have been developed around the idea that the generator in question is used to simulate a relatively simple stochastic system with *known* (population) performance measures that the simulation estimates. The simulated results are compared in some way with the known exact "answers," perhaps by means of a chi-square test. A simple application of this idea is seen in Prob. 7.10; Rudolph and Hawkins (1976) test several generators by using them to simulate Markov processes. In general, we feel that as many empirical tests should be performed as are practicable. In this regard, it should be mentioned that there are several comprehensive test suites for evaluating random-number generators. These include DIEHARD [Marsaglia (1995)], the NIST Test Suite [Rukhin et al. (2001)], and *TestU01* [L'Ecuyer and Simard (2007)], with the last package containing more than 100 empirical tests. These test suites are also described in Gentle (2010, pp. 79–85).

Lest the reader be left with the impression that the empirical tests we have presented in this section have no discriminative power at all, we subjected the infamous generator RANDU [defined by $Z_i = 65{,}539 Z_{i-1} (\text{mod } 2^{31})$], with seed $Z_0 = 123{,}456{,}789$, to the same tests as in Examples 7.6 through 7.9. The test statistics were as follows:

Chi-square test:	χ^2	= 4,202.0
Serial tests:	$\chi^2(2)$	= 4,202.3
	$\chi^2(3)$	= 16,252.3
Runs-up test:	R	= 6.3
Correlation tests:	All A_j's were insignificant	

While uniformity appears acceptable on [0, 1] and on the unit square, note the enormous value of the three-dimensional serial test statistic, indicating a severe problem for this generator in terms of uniformity on the unit cube. RANDU is a fatally flawed generator, due primarily to its utter failure in three dimensions; we shall see why in Sec. 7.4.2.

One potential disadvantage of empirical tests is that they are only *local*; i.e., only that segment of a cycle (for LCGs, for example) that was actually used to generate the U_i's for the test is examined, so we cannot say anything about how the generator might perform in other segments of the cycle. On the other hand, this local nature of empirical tests can be advantageous, since it might allow us to examine the actual random numbers that will be used later in a simulation. (Often we can calculate ahead of time how many random numbers will be used in a simulation, or at least get a conservative estimate, by analyzing the model's operation and the techniques used for generating the necessary random variates.) Then this entire random-number stream can be tested empirically, one would hope without excessive cost. (The tests in Examples 7.6 through 7.9 were all done together in a single program that took just a few seconds on an old, modest computer.) A more global empirical test could be performed by replicating an entire test several times and statistically comparing the observed values of the test statistics against the distribution under the null hypothesis; Fishman (1978, pp. 371–372) suggests this approach. For example, the runs-up test of Example 7.8 could be done, say, 100 times using 100 separate random-number streams from the same generator, each of length 5000. This would result in 100 independent values for R, which could then be compared with the chi-square distribution with 6 df using, for example, the K-S test with all parameters known.

7.4.2 Theoretical Tests

We now discuss theoretical tests for random-number generators. Since these tests are quite sophisticated and mathematically complex, we shall describe them somewhat qualitatively; for detailed accounts see Fishman (1996, pp. 607–628), Fishman (2006, pp. 119–123, 133–134, 140–141), Knuth (1998a, pp. 80–115), and L'Ecuyer (1998, pp. 106–114). As mentioned earlier, theoretical tests do not require that we generate any U_i's at all but are *a priori* in that they indicate how well a generator *can* perform by looking at its structure and defining constants. Theoretical tests also differ from empirical tests in that they are *global*; i.e., a generator's behavior over its *entire* cycle is examined. As we mentioned at the end of Sec. 7.4.1, it is debatable whether local or global tests are preferable; global tests have a natural appeal but do not generally indicate how well a specific segment of a cycle will behave.

It is sometimes possible to compute the "sample" mean, variance, and correlations over an entire cycle directly from the constants defining the generator. Many of these results are quoted by Kennedy and Gentle (1980, pp. 139–143). For example, in a full-period LCG, the average of the U_i's, taken over an entire cycle, is

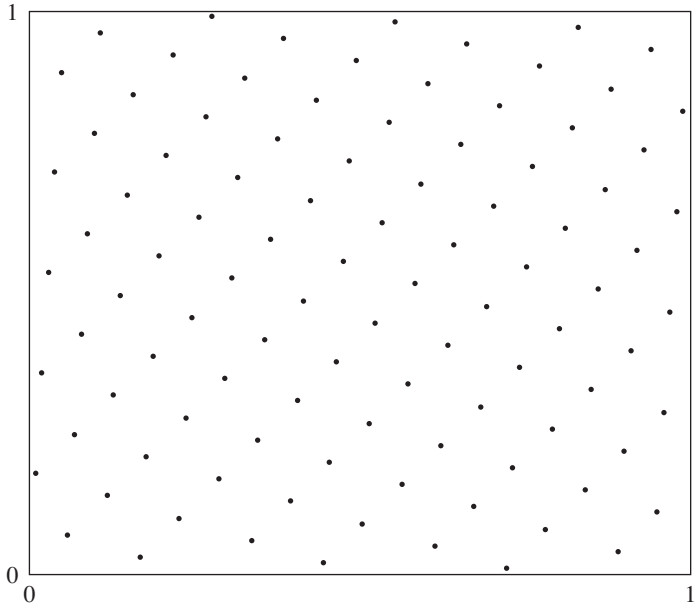


FIGURE 7.2
Two-dimensional lattice structure for the full-period LCG with $m = 101$ and $a = 18$.

$\frac{1}{2} - 1/(2m)$, which is seen to be very close to the desired $\frac{1}{2}$ if m is one of the large values (in the billions) typically used; see Prob. 7.14. Similarly, we can compute the "sample" variance of the U_i's over a full cycle and get $\frac{1}{12} - 1/(12m^2)$, which is close to $\frac{1}{12}$, the variance of the U(0, 1) distribution. Kennedy and Gentle also discuss "sample" correlations for LCGs. Although such formulas may seem comforting, they can be misleading; e.g., the result for the full-period LCG sample lag 1 correlation suggests that, to minimize this value, a be chosen close to $\sqrt{m}$, which turns out to be a poor choice from the standpoint of other important statistical considerations.

The best-known theoretical tests are based on the rather upsetting observation by Marsaglia (1968) that "random numbers fall mainly in the planes." That is, if U_1, $U_2, \ldots$ is a sequence of random numbers generated by a LCG, the overlapping d-tuples $(U_1, U_2, \ldots, U_d)$, $(U_2, U_3, \ldots, U_{d+1}), \ldots$ will all fall on a relatively small number of $(d-1)$-dimensional hyperplanes passing through the d-dimensional unit hypercube $[0, 1]^d$. For example, if $d = 2$, the pairs (U_1, U_2), $(U_2, U_3), \ldots$ will be arranged in a "lattice" fashion along several different families of parallel lines going through the unit square. The lines within a family are parallel to each other, but lines from different families are not parallel.

In Figs. 7.2 and 7.3, we display all possible pairs (U_i, U_{i+1}) for the full-period multiplicative LCGs $Z_i = 18Z_{i-1} (\text{mod } 101)$ and $Z_i = 2Z_{i-1} (\text{mod } 101)$, respectively.

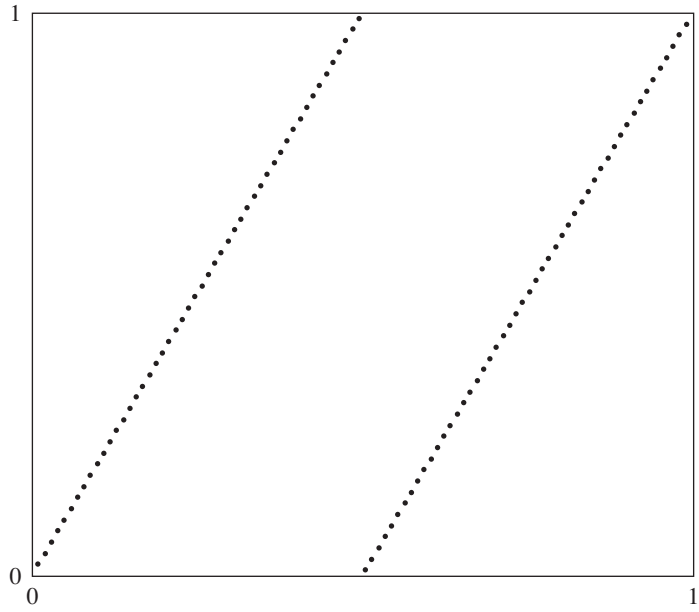


FIGURE 7.3
Two-dimensional lattice structure for the full-period LCG with $m = 101$ and $a = 2$.

(The period is 100 in each case, since the modulus $m = 101$ is prime and each value of the multiplier a is a primitive element modulo m.) While the apparent regularity in Fig. 7.2 certainly does not seem very "random," it may not be too disturbing since the pairs seem to fill up the unit square fairly well, or at least as well as could be expected with such a small modulus. On the other hand, all 100 pairs in Fig. 7.3 fall on just two parallel lines, which is definitely anxiety-provoking. Since there are large areas of the unit square where we can never realize a pair of U_i's, a simulation using such a generator would almost certainly produce invalid results.

The same difficulties occur in three dimensions. In Fig. 7.4, 2000 triples (U_i, U_{i+1}, U_{i+2}) produced by the infamous multiplicative LCG generator RANDU ($m = 2^{31}$ and $a = 65{,}539$) are displayed, viewed from a particular point outside of the unit cube. Note that all triples of U_i's fall on *only* 15 parallel planes passing through the unit cube. (In general, all the roughly half-billion triples across the period fall on these same planes.) This explains the horrific performance of RANDU on the three-dimensional serial test noted at the end of Sec. 7.4.1.

Among all families of parallel hyperplanes that cover all overlapping d-tuples $(U_i, U_{i+1}, \ldots, U_{i+d-1})$, take the one for which adjacent hyperplanes are farthest apart and denote this distance by $\delta_d(m, a)$. The idea of computing $\delta_d(m, a)$ was suggested by Coveyou and MacPherson (1967) and is typically called the *spectral test.* If $\delta_d(m, a)$ is small, then we would expect that the corresponding generator would be able to uniformly fill up the d-dimensional unit hypercube $[0, 1]^d$.

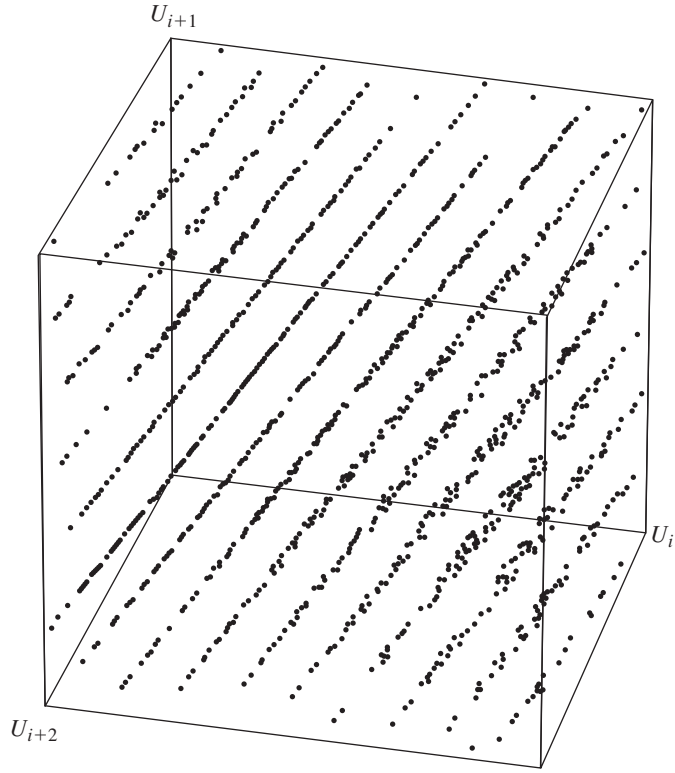


FIGURE 7.4
Three-dimensional lattice structure for 2000 triples from the multiplicative LCG RANDU with $m = 2^{31}$ and $a = 65{,}539$.

For LCGs it is possible to compute a theoretical lower bound $\delta_d^*(m)$ on $\delta_d(m, a)$, which is given by the following:

$$\delta_d(m, a) \geq \delta_d^*(m) = \frac{1}{\gamma_d m^{1/d}} \text{ for all } a$$

where γ_d is a constant whose exact value is known only for $d \leq 8$. We can then define the following figures of merit for a LCG:

$$S_d(m, a) = \frac{\delta_d^*(m)}{\delta_d(m, a)}$$

and

$$M_8(m, a) = \min_{2 \leq d \leq 8} S_d(m, a)$$

These figures of merit will be between 0 and 1, with values close to 1 being desirable. Methods for performing the spectral test [i.e., for computing $\delta_d(m, a)$] are discussed by Knuth (1998a, pp. 98–104) and L'Ecuyer and Couture (1997).

TABLE 7.5
The results of the spectral test for some LCGs

Figure of merit	Generator $m = 2^{31} - 1$ $a = a_1 = 16{,}807$	$m = 2^{31} - 1$ $a = a_2 = 630{,}360{,}016$	$m = 2^{31} - 1$ $a = 742{,}938{,}285$	$m = 2^{31}$ $a = 65{,}539$
$S_2(m, a)$	0.338	0.821	0.867	0.931
$S_3(m, a)$	0.441	0.432	0.861	0.012
$S_4(m, a)$	0.575	0.783	0.863	0.060
$S_5(m, a)$	0.736	0.802	0.832	0.157
$S_6(m, a)$	0.645	0.570	0.834	0.293
$S_7(m, a)$	0.571	0.676	0.624	0.453
$S_8(m, a)$	0.610	0.721	0.707	0.617
$M_8(m, a)$	0.338	0.432	0.624	0.012

Values of $S_d(m, a)$ and $M_8(m, a)$ are given in Table 7.5 for several well-known LCGs. The LCGs in columns 2 and 3 with multipliers a_1 and a_2 were discussed in Sec. 7.2.2, and it's seen that the latter generator is superior relative to the spectral test. The generator in column 4 with $a = 742{,}938{,}285$ was found by Fishman and Moore (1986) in an exhaustive search to be the best relative to the criterion $M_6(2^{31} - 1, a)$. Finally, in column 5 are the results of the spectral test for RANDU; its poor behavior in $d = 3$ dimensions is clearly reflected in the value $S_3(2^{31}, 65{,}539) = 0.012$.

7.4.3 Some General Observations on Testing

The number, variety, and range of complexity of tests for random-number generators are truly bewildering. To make matters worse, there has been (and probably always will be) considerable controversy over which tests are best, whether theoretical tests are really more definitive than empirical tests, and so on. Indeed, no amount of testing can ever absolutely convince everyone that some particular generator is absolutely "the best." One piece of advice that is often offered, however, is that a random-number generator should be tested in a way that is consistent with its intended use. This would entail, for example, examining the behavior of pairs of U_i's (perhaps with the two-dimensional serial test) if random numbers are naturally used in pairs in the simulation itself. In a broader sense, this advice would imply that one should be more careful in choosing and testing a random-number generator if the simulation in which it will be used is very costly, requires high-precision results, or is a particularly critical component of a larger study.

APPENDIX 7A
PORTABLE C CODE FOR A PMMLCG

Here we present computer code in C to implement the PMMLCG defined by modulus $m = m^* = 2^{31} - 1 = 2{,}147{,}483{,}647$ and multiplier $a = a_2 = 630{,}360{,}016$, discussed at the end of Sec. 7.2.2. The code shown here can be downloaded from www.mhhe.com/law. This code is based closely on the FORTRAN code of Marse and Roberts (1983), and it requires that integers between $-m^*$ and m^* be represented and computed correctly. This generator has 100 different streams that are spaced 100,000 apart.

It is generally *not* recommended that this generator be used for serious real-world applications, since the combined MRG in App. 7B has much better statistical properties.

Figure 7.5 gives code for an ANSI-standard C (i.e., using function prototyping) version of this generator, in three functions, as detailed in the comments. Figure 7.6 gives a header file (lcgrand.h) that the user must #include to declare the functions. We have used this code on a variety of computers and compilers, and it was used in the C examples in Chaps. 1 and 2.

```
/* Prime modulus multiplicative linear congruential generator
   Z[i] = (630360016 * Z[i-1]) (mod(pow(2,31) - 1)), based on Marse and Roberts'
   portable FORTRAN random-number generator UNIRAN.  Multiple (100) streams are
   supported, with seeds spaced 100,000 apart.  Throughout, input argument
   "stream" must be an int giving the desired stream number.  The header file
   lcgrand.h must be included in the calling program (#include "lcgrand.h")
   before using these functions.

   Usage: (Three functions)

   1. To obtain the next U(0,1) random number from stream "stream," execute
          u = lcgrand(stream);
      where lcgrand is a float function.   The float variable u will contain the
      next random number.

   2. To set the seed for stream "stream" to a desired value zset, execute
          lcgrandst(zset, stream);
      where lcgrandst is a void function and zset must be a long set to the
      desired seed, a number between 1 and 2147483646 (inclusive).  Default
      seeds for all 100 streams are given in the code.

   3. To get the current (most recently used) integer in the sequence being
      generated for stream "stream" into the long variable zget, execute
          zget = lcgrandgt(stream);
      where lcgrandgt is a long function. */

/* Define the constants. */

#define MODLUS 2147483647
#define MULT1       24112
#define MULT2       26143

/* Set the default seeds for all 100 streams. */
```

FIGURE 7.5
C code for the PMMLCG with $m = 2^{31} - 1$ and $a = 630{,}360{,}016$ based on Marse and Roberts (1983).

```
static long zrng[] =
{         1,
 1973272912, 281629770,  20006270,1280689831,2096730329,1933576050,
  913566091, 246780520,1363774876, 604901985,1511192140,1259851944,
  824064364, 150493284, 242708531,  75253171,1964472944,1202299975,
  233217322,1911216000, 726370533, 403498145, 993232223,1103205531,
  762430696,1922803170,1385516923,  76271663, 413682397, 726466604,
  336157058,1432650381,1120463904, 595778810, 877722890,1046574445,
   68911991,2088367019, 748545416, 622401386,2122378830, 640690903,
 1774806513,2132545692,2079249579,  78130110, 852776735,1187867272,
 1351423507,1645973084,1997049139, 922510944,2045512870, 898585771,
  243649545,1004818771, 773686062, 403188473, 372279877,1901633463,
  498067494,2087759558, 493157915, 597104727,1530940798,1814496276,
  536444882,1663153658, 855503735,  67784357,1432404475, 619691088,
  119025595, 880802310, 176192644,1116780070, 277854671,1366580350,
 1142483975,2026948561,1053920743, 786262391,1792203830,1494667770,
 1923011392,1433700034,1244184613,1147297105, 539712780,1545929719,
  190641742,1645390429, 264907697, 620389253,1502074852, 927711160,
  364849192,2049576050, 638580085, 547070247 };
/* Generate the next random number. */

float lcgrand(int stream)
{
    long zi, lowprd, hi31;

    zi     = zrng[stream];
    lowprd = (zi & 65535) * MULT1;
    hi31   = (zi >> 16) * MULT1 + (lowprd >> 16);
    zi     = ((lowprd & 65535) - MODLUS) +
             ((hi31 & 32767) << 16) + (hi31 >> 15);
    if (zi < 0) zi += MODLUS;
    lowprd = (zi & 65535) * MULT2;
    hi31   = (zi >> 16) * MULT2 + (lowprd >> 16);
    zi     = ((lowprd & 65535) - MODLUS) +
             ((hi31 & 32767) << 16) + (hi31 >> 15);
    if (zi < 0) zi += MODLUS;
    zrng[stream] = zi;
    return (zi >> 7 | 1) / 16777216.0;
}


void lcgrandst (long zset, int stream) /* Set the current zrng for stream
                                          "stream" to zset. */
{
    zrng[stream] = zset;
}


long lcgrandgt (int stream) /* Return the current zrng for stream "stream". */
{
    return zrng[stream];
}
```

FIGURE 7.5
(*continued*)

```
/* The following 3 declarations are for use of the random-number generator
   lcgrand and the associated functions lcgrandst and lcgrandgt for seed
   management.  This file (named lcgrand.h) should be included in any program
   using these functions by executing
       #include "lcgrand.h"
   before referencing the functions. */

float lcgrand(int stream);
void  lcgrandst(long zset, int stream);
long  lcgrandgt(int stream);
```

FIGURE 7.6
C header file (lcgrand.h) to accompany the C code in Fig. 7.5.

APPENDIX 7B
PORTABLE C CODE FOR A COMBINED MRG

Here we present an ANSI-standard C function, mrand, that implements the combined MRG specified in Sec. 7.3.2, taken from L'Ecuyer (1999a), and which supports multiple streams (up to 10,000) with seed vectors spaced 10^{16} (ten quadrillion) apart. This code requires that all integers between -2^{53} and 2^{53} be represented exactly in floating point, which will be satisfied in the (common) situation of a 32-bit word-length machine and a C compiler that conforms to the IEEE standard for floating-point storage and arithmetic. All codes shown here can be downloaded from www.mhhe.com/law.

Figure 7.7 gives the code for this generator, in three functions, as described in the comments in the code. Figure 7.8 gives a header file (mrand.h) that the user must #include before the main function in the calling program. Figure 7.9 gives the first 23 and last 4 lines (showing the seed vectors for streams 1–20 and 9998–10,000) of the file mrand_seeds.h, which contains seed vectors for the 10,000 streams spaced 10^{16}

```
/* Combined MRG from Sec. 7.3.2, from L'Ecuyer (1999a).  Multiple
   (10,000) streams are supported, with seed vectors spaced
   10,000,000,000,000,000 apart.  Throughout, input argument "stream"
   must be an int giving the desired stream number.  The header file
   mrand_seeds.h is included here, so must be available in the
   appropriate directory.  The header file mrand.h must be included in
   the calling program (#include "mrand.h") before using these
   functions.

   Usage: (Three functions)

   1. To obtain the next U(0,1) random number from stream "stream,"
      execute
          u = mrand(stream);
      where mrand is a double function.  The double variable u will
      contain the next random number.

   2. To set the seed vector for stream "stream" to a desired 6-vector,
      execute
          mrandst(zset, stream);
      where mrandst is a void function and zset must be a double
      vector with positions 0 through 5 set to the desired
      stream 6-vector, as described in Sec. 7.3.2.

   3. To get the current (most recently used) 6-vector of integers in
      the sequences (to use, e.g., as the seed for a subsequent
      independent replication), into positions 0 through 5 of the
      double vector zget, execute
          mrandgt(zget, stream);
      where mrandgt is void function.  */

#include "mrand_seeds.h"
#define norm   2.328306549295728e-10  /* 1.0/(m1+1) */
#define norm2  2.328318825240738e-10  /* 1.0/(m2+1) */
#define m1     4294967087.0
#define m2     4294944443.0

/* Generate the next random number. */
```

FIGURE 7.7
C code for the combined MRG specified in Sec. 7.3.2, based on L'Ecuyer (1999a).

```
double mrand(int stream)
{
    long k;
    double p,
           s10 = drng[stream][0], s11 = drng[stream][1], s12 = drng[stream][2],
           s20 = drng[stream][3], s21 = drng[stream][4], s22 = drng[stream][5];

    p = 1403580.0 * s11 - 810728.0 * s10;
    k = p / m1;  p -= k*m1;  if (p < 0.0) p += m1;
    s10 = s11;   s11 = s12;  s12 = p;

    p = 527612.0 * s22 - 1370589.0 * s20;
    k = p / m2;  p -= k*m2;  if (p < 0.0) p += m2;
    s20 = s21;   s21 = s22;  s22 = p;

    drng[stream][0] = s10;  drng[stream][1] = s11;  drng[stream][2] = s12;
    drng[stream][3] = s20;  drng[stream][4] = s21;  drng[stream][5] = s22;

    if (s12 <= s22) return ((s12 - s22 + m1) * norm);
    else return ((s12 - s22) * norm);
}

/* Set seed vector for stream "stream". */

void mrandst(double* seed, int stream)
{
int i;
    for (i = 0; i <= 5; ++i) drng[stream][i] = seed[i];
}

/* Get seed vector for stream "stream". */

void mrandgt(double* seed, int stream)
{
int i;
    for (i = 0; i <= 5; ++i) seed[i] = drng[stream][i];
}
```

FIGURE 7.7
(*continued*)

```
/* Header file "mrand.h" to be included by programs using mrand.c */

double mrand(int stream);
void mrandst(double* seed, int stream);
void mrandgt(double* seed, int stream);
```

FIGURE 7.8
C header file (mrand.h) to accompany the C code in Fig. 7.7.

apart. We have used these codes on a variety of computers and compilers successfully, though some compilers might issue harmless warnings about the size of the numbers in mrand.h (such warnings can usually be turned off by a compiler switch like -w on some UNIX C compilers). Also, the calling program might have to load a mathematical library (e.g., the -lm switch on UNIX C compilers).

Codes for this generator in C, C++, and Java, which allow for substreams (see Sec. 7.3.2), can be downloaded from www.iro.umontreal.ca/~lecuyer. [See also L'Ecuyer, Simard, Chen, and Kelton (2002).]

```
/* Header file "mrand_seeds.h" included by mrand.c */

static double drng[][6] =
{
           0,           0,           1,           0,           0,           1,
  1772212344,  1374954571,  2377447708,   540628578,  1843308759,   549575061,
  2602294560,  1764491502,  3872775590,  4089362440,  2683806282,   437563332,
   376810349,  1545165407,  3443838735,  3650079346,  1898051052,  2606578666,
  1847817841,  3038743716,  2014183350,  2883836363,  3242147124,  1955620878,
  1075987441,  3468627582,  2694529948,   368150488,  2026479331,  2067041056,
   134547324,  4246812979,  1700384422,  2358888058,    83616724,  3045736624,
  2816844169,   885735878,  1824365395,  2629582008,  3405363962,  1835381773,
   675808621,   434584068,  4021752986,  3831444678,  4193349505,  2833414845,
  2876117643,  1466108979,   163986545,  1530526354,    68578399,  1111539974,
   411040508,   544377427,  2887694751,   702892456,   758163486,  2462939166,
  3631741414,  3388407961,  1205439229,   581001230,  3728119407,    94602786,
  4267066799,  3221182590,  2432930550,   813784585,  1980232156,  2376040999,
  1601564418,  2988901653,  4114588926,  2447029331,  4071707675,  3696447685,
  3878417653,  2549122180,  1351098226,  3888036970,  1344540382,  2430069028,
   197118588,  1885407936,   576504243,   439732583,   103559440,  3361573194,
  4024454184,  2530169746,  2135879297,  2516366026,   260078159,  2905856966,
  2331743881,  2059737664,   186644977,   401315249,    72328980,  1082588425,
   694808921,  2851138195,  1756125381,  1738505503,  2662188364,  3598740668,
  2834735415,  2017577369,  3257393066,  3823680297,  2315410613,   637316697,
  4132025555,  3700940887,   838767760,  2818574268,  1375004287,  2172829019,
.
.
.
   560024289,  1830276631,   144885590,  1556615741,  1597610225,  1856413969,
  1031792556,  1844191084,  1441357589,  3147919604,   199001354,  2555043119,
  2023049680,  4184669824,  4074523931,   252765086,  3328098427,  1480103038
};
```

FIGURE 7.9
Excerpt from seed-vector file (mrand_seeds.h) to accompany the C codes in Figs. 7.7 and 7.8 (complete file can be downloaded from www.mhhe.com/law).

PROBLEMS

7.1. For the LCG of Example 7.2 find Z_{500}, using only pencil and paper.

7.2. For the following multiplicative LCGs, compute Z_i for enough values of $i \geq 1$ to cover an entire cycle:
(a) $Z_i = (11Z_{i-1}) \pmod{16}$, $Z_0 = 1$
(b) $Z_i = (11Z_{i-1}) \pmod{16}$, $Z_0 = 2$
(c) $Z_i = (2Z_{i-1}) \pmod{13}$, $Z_0 = 1$
(d) $Z_i = (3Z_{i-1}) \pmod{13}$, $Z_0 = 1$

Note that (a) and (b) have m of the form 2^b; (c) is a PMMLCG, for which $a = 2$ is a primitive element modulo $m = 13$.

7.3. Without actually computing any Z_i's, determine which of the following mixed LCGs have full period:
(a) $Z_i = (13Z_{i-1} + 13) \pmod{16}$
(b) $Z_i = (12Z_{i-1} + 13) \pmod{16}$
(c) $Z_i = (13Z_{i-1} + 12) \pmod{16}$
(d) $Z_i = (Z_{i-1} + 12) \pmod{13}$

7.4. For the four mixed LCGs in Prob. 7.3, compute Z_i for enough values of $i \geq 1$ to cover an entire cycle; let $Z_0 = 1$ in each case. Comment on the results.

7.5. (*a*) Implement the PMMLCG $Z_i = 630{,}360{,}016 Z_{i-1} (\mathrm{mod}\, 2^{31} - 1)$ on your computer, using the Marse-Roberts code in App. 7A.
(*b*) Repeat the empirical tests of Examples 7.6 through 7.9 with this generator, and compare your results with those given in the examples.

7.6. Use the multiplicative LCG in part (*a*) of Prob. 7.2 to shuffle the output from the mixed LCG in part (*d*) of Prob. 7.3, using a vector **V** of length 2. Let the seed for both LCGs be 1, and list the first 100 values of **V**, I, and V_I. Identify the period and comment generally on the results.

7.7. For the chi-square test of uniformity, verify the following algorithm for computing $f_1, f_2, \ldots, f_k$:

```
Set f_j = 0 for j = 1,2, . . . , k
For i = 1, . . . , n do
   Generate U_i
   Set J = ⌈kU_i⌉
   Replace f_J by f_J + 1
End do
```

(For a real number x, $\lceil x \rceil$ denotes the smallest integer that is greater than or equal to x.) Generalize this algorithm to compute the test statistic for a general d-dimensional serial test.

7.8. Show that the following algorithm correctly computes $r_1, r_2, \ldots, r_6$ for the runs-up test, from the generated numbers $U_1, U_2, \ldots, U_n$:

```
Set r_j = 0 for j = 1, . . . , 6
Generate U_1, set A = U_1, and set J = 1
For i = 2, . . . , n do
   Generate U_i and set B = U_i
   If A ≥ B then
      Set J = min(J,6)
      Replace r_J by r_J + 1
      Set J = 1
   Else
      Replace J by J + 1
   End if
   Replace A by B
End do
Set J = min(J,6)
Replace r_J by r_J + 1
```

7.9. Subject the canned random-number generator on your computer to the chi-square test, two- and three-dimensional serial tests, the runs-up test, and correlation tests at lags $1, 2, \ldots, 5$. Use the same values for n, k, and α that were used in Examples 7.6 through 7.9. (If your generator does not pass these tests, we suggest that you exercise caution in using it until you can obtain more information on it, either from the literature or from your own further testing.)

7.10. A general approach to testing a random-number generator empirically is to use it to simulate a *simple* stochastic model and obtain estimates of *known* parameters; a standard test is then used to compare the estimate(s) against the known parameter(s). For

example, we know that in throwing two fair dice independently, the sum of the two outcomes will be 2, 3, . . . , 12 with respective probabilities $\frac{1}{36}$, $\frac{1}{18}$, $\frac{1}{12}$, $\frac{1}{9}$, $\frac{5}{36}$, $\frac{1}{6}$, $\frac{5}{36}$, $\frac{1}{9}$, $\frac{1}{12}$, $\frac{1}{18}$, and $\frac{1}{36}$. Simulate 1000 independent throws of a pair of independent fair dice, and compare the observed proportion of 2s, 3s, . . . , 12s with the known probabilities, using an appropriate test from Chap. 6. Use the canned generator on your computer or one of the generators in App. 7A or 7B.

7.11. For the LCG in part (*d*) of Prob. 7.3, plot the pairs (U_1, U_2), (U_2, U_3), . . . and observe the lattice structure obtained. Note that this LCG has full period.

7.12. Consider the Fibonacci generator discussed in Sec. 7.3.1.

(*a*) Show that this generator can never produce the following arrangement of three consecutive output values: $U_{i-2} < U_i < U_{i-1}$.

(*b*) Show that the arrangement in (*a*) should occur with probability $\frac{1}{6}$ for a "perfect" random-number generator.

This points out a gross defect in the generator, as noted by Bratley, Fox, and Schrage (1987), who credit U. Dieter with having noticed this.

7.13. Suppose that $U_1, U_2, \ldots, U_k$ are IID U(0, 1) random variables. Show that the fractional part (i.e., ignoring anything to the left of the decimal point) of $U_1 + U_2 + \cdots + U_k$ is also uniformly distributed on the interval [0, 1]. [This property motivated the composite generator of Wichmann and Hill (1982).]

7.14. Show that the average of the U_i's taken over an entire cycle of a full-period LCG is $\frac{1}{2} - 1/(2m)$. [*Hint:* for a positive integer k, *Euler's formula* states that $1 + 2 + \cdots + k = k(k + 1)/2$.]

7.15. For Example 7.3, use Eq. (7.6) to show that the period of the W_i's is 31.